
CS689: Machine Learning - Fall 2024

Final Exam: Solutions

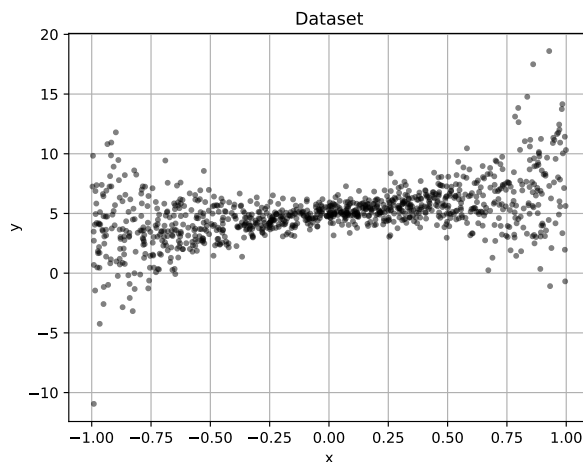
Dec 12, 2024

Name: _____

Instructions: Write your name on the exam sheet. No electronic devices may be used during the exam. You may consult your paper notes and/or a print copy of texts during the exam. Sharing of notes/texts during the exam is strictly prohibited. Show your work for all derivation questions. A suitable explanation must be provided for all questions to earn full credit. Attempt all problems. Partial credit may be given for partially correct or incomplete answers. If your answer to a question spans two pages, please make sure to indicate that at the end of the first page of the answer. If you have questions at any time, please raise your hand.

Problem	Topic	Page	Points
1	Probabilistic Regression Modeling	1	10
2	Probabilistic Regression Implementation	3	10
3	MLPs vs CNNs	5	10
4	Joint Distributions	7	10
5	Mixture of Experts	9	10
6	Mixture Models vs Factor Analysis	11	10
7	Factor Analysis Learning	13	10
8	RNNs vs Transformers	15	10
9	Autoencoders	17	10
10	Variational Autoencoders	19	10

1. (10 points) Probabilistic Regression Modeling: Consider the data set shown below. Suppose we are trying to model these data using a conditional normal model of the form $p(Y = y|X = x) = \mathcal{N}(y; \mu(x), (\sigma(x))^2)$ where $y \in \mathbb{R}$ and $x \in \mathbb{R}$. To make learning as efficient as possible, we want to specify functional forms for $\mu(x)$ and $\sigma(x)$ that will provide an accurate fit to the data set while having the minimum possible complexity. What functional forms should we choose for $\mu(x)$ and $\sigma(x)$? Explain your answer.



a. (5 pts) Choice of functional form and explanation for $\mu(x)$.

Example Solution: We should select $\mu(x) = ax + b$. As we can see from the data set, the mean relationship between x and y appears to be linear, so a linear function is the minimum complexity choice that should fit the data set well.

b. (5 pts) Choice of functional form and explanation for $\sigma(x)$.

Example Solution: We should select $\sigma(x) = \log(1 + \exp(cx^2 + dx + f))$. As we can see, the spread of the data seems to increase symmetrically and faster than linearly as x gets farther from 0. At the same time, the standard deviation function must give a positive value of all x . To satisfy this constraint, match the structure of the data set, and have minimal complexity, we apply the softplus function to a quadratic function of x .

2. (10 points) Probabilistic Regression Implementation: A student is tasked with implementing a probabilistic regression model based on the exponential distribution $p(Y = y) = b \exp(-b \cdot y)$ with $y, b \in \mathbb{R}^{>0}$. The student produces the PyTorch implementation shown below where the forward function is intended to compute $\log p(Y = y|X = x)$ for use in learning the model parameters. What major mistake did the student make? Explain your answer.

```
import torch
import torch.nn as nn

class expreg(nn.Module):
    def __init__(self,D):
        super().__init__()
        self.fc = nn.Linear(D,1)
    def forward(self,y,x):
        out = self.fc(x)
        return torch.log(out) - out*y
```

Example Solution: As we can see from the specification of $p(Y = y) = b \exp(-b \cdot y)$, the b term must be non-negative. When we construct a probabilistic regression model from this distribution, it will have the form $p(Y = y|X = x) = b(x) \exp(-b(x) \cdot y)$, but the function $b(x)$ must satisfy $b(x) > 0$ for any x . From the implementation, we can see that the function $b(x)$ was chosen to be linear via use of the `nn.Linear` module to compute `out=self.fc(x)`. A linear function of x can produce both positive or negative output values depending on x and the parameters. This means the implementation is not enforcing the required parameter constraint in the forward function, which will lead learning to fail.

3. (10 points) MLPs vs CNNs: Consider the problem of hyperspectral image classification. A hyperspectral image represents the intensity of many frequency bands of light instead of just the red, green and blue components represented by standard RGB images. A hyperspectral image is represented as a tensor of shape (H, W, F) where H is the height of the image, W is the width of the image and F is the number of frequency bands. Answer the following questions.

a. (5 pts) Explain why the number of parameters in the first layer of an MLP for hyperspectral image classification must scale with H , W and F .

Example Solution: In a standard MLP architecture, the first layer of the model is a fully connected linear layer that maps a $(D, 1)$ shape vector into a hidden representation $\mathbf{h}$ of length K of our choice. To apply an MLP to this problem, we need to flatten the images tensors into vectors. The flattened images will have shape $(HWF, 1)$. Since the fully connected linear layer has one weight parameter for each pair of hidden and input dimension, the number of weight parameters will be $HWFK$, which scales with H , W , and F . The bias terms will contribute an additional K parameters.

b. (5 pts) Explain why the number of parameters in the first layer of a CNN for hyperspectral image classification only needs to scale with F .

Example Solution: The first layer in a convolutional neural network is a convolutional layer. A convolutional layer consists of any number K of convolution kernels. Each kernel is chosen to have height C and width C for some value C that is independent of the width and height of the input image. However, a convolution kernel combines information from all of its input channels. A convolution kernel is thus represented by a tensor of weights of shape (C, C, F) and one bias, thus the total number of parameters in a convolution layer is $(C^2F + 1)K$, which scales with F , but not H or W .

4. (10 points) Joint Distributions: Consider a joint distribution over D binary random variables $P(\mathbf{X} = \mathbf{x})$ where $\mathbf{x} = [x_1, \dots, x_D]$ and for each $1 \leq d \leq D$ we have $x_d \in \{0, 1\}$. Answer the following questions.

a. (5 pts) Suppose the model is parameterized such that $P(\mathbf{X} = \mathbf{x}) = \theta_{\mathbf{x}}$. What is the optimal computational complexity when computing a marginal distribution over K of the D variables? Explain your answer and give supporting equations.

Example Solution: Given $D - K$ binary variables to marginalize over in a distribution with no structure, the best case computational complexity is $O(2^{D-K})$. If we partition the variables into an index set A containing the K variables whose marginal we want to obtain and an index set B of the $D - K$ variables that we need to marginalize over, the equation below specifies the required marginalization operation. As we can see, we need to sum the joint probability mass function over all joint configurations of the $D - K$ variables. There are 2^{D-K} total configurations of these variables, so the sum requires $O(2^{D-K})$ time to compute.

$$P(\mathbf{X}_A = \mathbf{x}_A) = \sum_{x_{b_1}=0}^1 \cdots \sum_{x_{b_{D-K}}=0}^1 P(\mathbf{X}_A = \mathbf{x}_A, \mathbf{X}_B = \mathbf{x}_B)$$

b. (5 pts) Under the assumption that $P(\mathbf{X} = \mathbf{x})$ can be represented as a product of marginals $P(\mathbf{X} = \mathbf{x}) = \prod_{d=1}^D P(X_d = x_d)$, what is the optimal computational complexity when computing a marginal distribution over K of the D variables? Explain your answer and give supporting equations.

Example Solution: If the distribution can be represented as a product of marginals, then no computation is required to compute the marginal distribution itself on K of the D variables. If we partition the variables into an index set A containing the K variables whose marginal we want to obtain and an index set B of the $D - K$ variables that we need to marginalize over, the marginal distribution is obtained as a product over the K variables in the index set A . This result follows from the fact that we can isolate and marginalize over the variables in B one at a time due to the product form of the joint distribution.

$$P(\mathbf{X}_A = \mathbf{x}_A) = \prod_{i=1}^K P(X_{A_i} = x_{A_i})$$

However, while it takes no work to obtain the marginal distribution, computing an actual marginal probability on K variables requires K multiplications as shown above and thus has complexity $O(K)$.

5. (10 points) Mixture of Experts: A mixture of experts is a type of mixture model primarily used for supervised learning. The structured model specified below is for a mixture of linear regression experts. We have $z \in \{1, \dots, K\}$, $\mathbf{x} \in \mathbb{R}^D$, and $y \in \mathbb{R}$. Consider this model when answering the following questions.

$$\begin{aligned} P(Z = z) &= \pi_z \\ p(\mathbf{X} = \mathbf{x} | Z = z) &= \mathcal{N}(\mathbf{x}; \mu_z, \Sigma_z) \\ p(Y = y | \mathbf{X} = \mathbf{x}, Z = z) &= \mathcal{N}(y; \mathbf{x}\mathbf{w}_z, \sigma_z^2) \\ P(Z = z, \mathbf{X} = \mathbf{x}, Y = y) &= P(Z = z)p(\mathbf{X} = \mathbf{x} | Z = z)p(Y = y | \mathbf{X} = \mathbf{x}, Z = z) \end{aligned}$$

a. (5 pts) Derive an expression for $p(Y = y)$ in terms of the model components. Simplify to the extent possible. Show your work and briefly explain the key steps in your answer.

Example Solution: This is a marginal distribution on Y . We can obtain it starting from the joint distribution and marginalizing over Z and $\mathbf{X}$ using summation and integration respectively. The resulting expression does not simplify further.

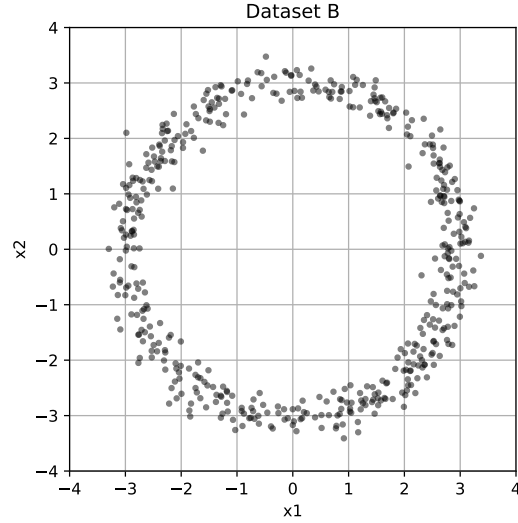
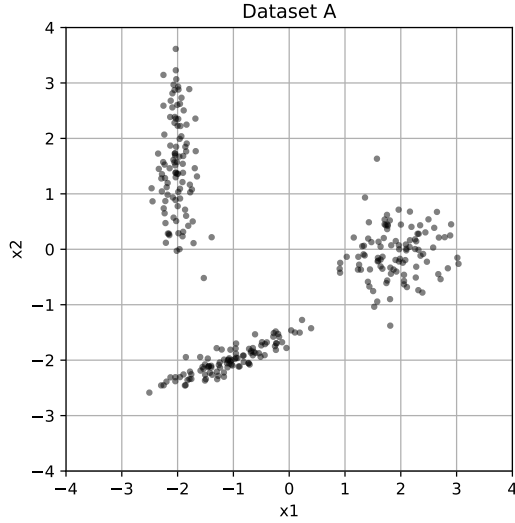
$$\begin{aligned} p(Y = y) &= \sum_{z=1}^K \int P(Z = z, \mathbf{X} = \mathbf{x}, Y = y) d\mathbf{x} \\ &= \sum_{z=1}^K \int \pi_z \cdot \mathcal{N}(\mathbf{x}; \mu_z, \Sigma_z) \cdot \mathcal{N}(y; \mathbf{x}\mathbf{w}_z, \sigma_z^2) d\mathbf{x} \end{aligned}$$

b. (5 pts) Derive an expression for $p(Y = y | \mathbf{X} = \mathbf{x})$ in terms of the model components. Simplify to the extent possible. Show your work and briefly explain the key steps in your answer.

Example Solution: We need to apply the definition of conditioning, and then derive the marginal distributions required in the numerator and denominator. The resulting expression does not simplify further.

$$\begin{aligned} p(Y = y | \mathbf{X} = \mathbf{x}) &= \frac{p(Y = y, \mathbf{X} = \mathbf{x})}{p(\mathbf{X} = \mathbf{x})} \\ &= \frac{\sum_{z=1}^K P(Z = z, \mathbf{X} = \mathbf{x}, Y = y)}{\sum_{z'=1}^K \int P(Z = z', \mathbf{X} = \mathbf{x}, Y = y') dy'} \\ &= \frac{\sum_{z=1}^K \pi_z \cdot \mathcal{N}(\mathbf{x}; \mu_z, \Sigma_z) \cdot \mathcal{N}(y; \mathbf{x}\mathbf{w}_z, \sigma_z^2)}{\sum_{z'=1}^K \int \pi_{z'} \cdot \mathcal{N}(\mathbf{x}; \mu_{z'}, \Sigma_{z'}) \cdot \mathcal{N}(y'; \mathbf{x}\mathbf{w}_{z'}, \sigma_{z'}^2) dy'} \end{aligned}$$

6. (10 points) Mixture Models vs Factor Analysis: Consider the two data sets shown below. For each data set, state whether the mixture of multivariate normals model or the factor analysis model would provide a better model. Explain each answer.



a. (5 pts) Data set A:

Example Solution: The mixture of multivariate normals model is the better choice. The mixture of multivariate normals model is designed to model multiple clusters of data where each cluster has a multivariate normal density. In 2D, the multivariate normal density has elliptical contours that can have any center, scale and orientation. This matches exactly data set A using $K = 3$ clusters. On the other hand, the linear factor analysis model has a marginal distribution in the form of a single multivariate normal. It thus can not fit these data well since the data set has clusters.

b. (5 pts) Data set B:

Example Solution: The mixture of multivariate normals model is the better choice. The factor analysis model is designed to model manifolds, and this data set corresponds to samples from a 1D manifold (a circle). However, the factor analysis model can only represent data sampled from a linear manifold of the form $\mathbf{W}\mathbf{z}$. If $\mathbf{z} \in \mathbf{R}^1$, this is a space of straight lines in $\mathbf{R}^2$. If $\mathbf{z} \in \mathbf{R}^2$, then $\mathbf{W}\mathbf{z}$ spans all of $\mathbf{R}^2$. As a result, the factor analysis model can not represent data sampled from the given circular manifold. On the other hand, by making K large enough, we can approximate the circular density for this data set arbitrarily well. The model can do this by tiling the circle with many overlapping multivariate normal distributions.

7. (10 points) Factor Analysis Learning: Recall that factor analysis has the marginal probability distribution shown below where $\mathbf{x} \in \mathbb{R}^D$. Describe an approach to defining the optimization variables and computing the negative log marginal likelihood (NLML) that will support direct NLML minimization via automatic differentiation while ensuring numerical stability for large D . Explain your design choices and provide supporting equations to specify your proposed computations. (Note: this question is not asking for a code-level implementation.)

$$p(\mathbf{X} = \mathbf{x}) = \mathcal{N}(\mathbf{x}; 0, \mathbf{w}\mathbf{w}^T + \psi)$$

$$= \frac{1}{|2\pi(\mathbf{w}\mathbf{w}^T + \psi)|^{1/2}} \exp\left(-\frac{1}{2}\mathbf{x}^T(\mathbf{w}\mathbf{w}^T + \psi)^{-1}\mathbf{x}\right)$$

Example Solution: First of all, to support direct NLML minimization, we need to respect the parameter constraint on ψ . ψ must be a positive diagonal matrix. We will represent ψ using a vector of real values η . We will define $\psi_{dd} = \log(1 + \exp(\eta_d))$ to ensure positivity of the diagonal elements via the soft plus function. We will fix $\psi_{dd'} = 0$ for $d \neq d'$ to ensure ψ is a diagonal matrix. $\mathbf{w}$ is not constrained and can be used as is. The parameters are this $\theta = [\mathbf{w}, \eta]$.

To ensure numerical stability, we need to compute the NLML explicitly in log space. This leads to the following result, with ψ computed as defined above.

$$NLML(\mathcal{D}, \theta) = \frac{N}{2} \log(|2\pi(\mathbf{w}\mathbf{w}^T + \psi)|) + \frac{1}{2} \sum_{n=1}^N \mathbf{x}_n^T (\mathbf{w}\mathbf{w}^T + \psi)^{-1} \mathbf{x}_n$$

The other numerical stability issues that we need to account for are the computation of the log of the determinant and the computation of the matrix inverse. The log determinant can be computed safely using the Cholesky factorization via the logdet function. The matrix inverse can similarly be computed based on the Cholesky factorization or any similar matrix decomposition such as the eigendecomposition. The result of $(\mathbf{w}\mathbf{w}^T + \psi)^{-1}\mathbf{x}$ can also be found in a numerically stable way by solving the linear system $\mathbf{v}(\mathbf{w}\mathbf{w}^T + \psi) - \mathbf{x} = 0$ for $\mathbf{v}$, and then computing $\mathbf{x}^T(\mathbf{w}\mathbf{w}^T + \psi)^{-1}\mathbf{x}$ as $\mathbf{x}^T\mathbf{v}$.

8. (10 points) RNNs vs Transformers: Both RNNs and the transformer self-attention layer have the ability to produce a hidden representation for each element in a sequence of arbitrary length using a number of parameters that is independent of the sequence length. Explain two advantages of the self-attention layer compared to RNNs when computing hidden representations for sequences.

a. (5 pts) Specify and explain advantage 1.

Example Solution: RNNs are fundamentally sequential models. They must perform the computations required to compute the hidden state of sequence position t before computing the hidden state for sequence position $t + 1$. By contrast, the transformer self-attention layer can compute the hidden representation for all sequence positions in parallel.

b. (5 pts) Specify and explain advantage 2.

Example Solution: RNNs have significant difficulty learning dependencies between distant sequence positions t and t' . This is because the RNN has to hold on to a latent representation of the input at position t for many time steps in order for that information to be used at position t' and that information risks being degraded by the processing of other inputs between t and t' . By contrast, when producing a hidden representation for a given sequence position t , the self-attention computation can make immediate use of values from any other sequence position t' regardless of how far t is from t' so long as it can learn to attend to position t' .

9. (10 points) Autoencoders: Suppose we conduct a survey consisting of a large number D of multiple choice questions. Let C_d be the number of multiple choice answers for question d . Each completed survey can be represented as a vector of categorical values $\mathbf{x} = [x_1, \dots, x_D]$ with $x_d \in \{1, \dots, C_d\}$. Suppose N people complete the survey, resulting in a data set $\mathcal{D} = \{\mathbf{x}_n | 1 \leq n \leq N\}$. Suppose we want to learn a non-linear autoencoder to reduce the dimensionality of the data set of survey responses to $K = 2$ dimensions to better visualize the data. Answer the following questions.

a. (5 pts) How would you represent the data when learning the model and what loss function would you use? Provide supporting equations and explain your answer.

Example Solution: Since the data are categorical, using a reconstruction loss that compares the numerical values of the categories generally does not make sense. One approach to overcome this issue is to use a “one-hot” encoding of each categorical variable. This replaces each single categorical value x_d with a binary vector $\mathbf{v}_d$ of length C_d with a 1 in position x_d and zeroes elsewhere (e.g., if $x_1 = 2$ and $C_1 = 3$, $\mathbf{v}_1 = [0, 1, 0]$). We can then form new data vectors $\mathbf{v} = [\mathbf{v}_1, \dots, \mathbf{v}_D]$ by concatenating the one-hot representations. Finally, we can use the standard squared reconstruction loss applied to these vectors and their reconstructions to learn the model. The learning objective is shown below where f_θ represents the output of the full autoencoder (encoder and decoder).

$$L(\mathbf{D}, \theta) = \sum_{n=1}^N \|\mathbf{v}_n - f_\theta(\mathbf{v}_n)\|_2^2$$

b. (5 pts) How could you apply the denoising principle to this learning problem? Provide supporting equations and explain your answer.

Example Solution: To apply the denoising principle to this problem, we could apply random Bernoulli noise to the $\mathbf{v}$ vectors used as input to the autoencoder; however, this will immediately break the one-hot property. A better choice is to apply categorical noise to the $\mathbf{x}$ vectors, obtaining noisy categorical vectors $\mathbf{x}'$, and then to convert the $\mathbf{x}'$ vectors to one-hot vectors $\mathbf{v}'$. To apply categorical noise, we can decide with some probability p whether to corrupt each dimensions d . If we decide to corrupt dimension d , we can select a new value uniformly at random from the $C_d - 1$ values that are not the true value x_d . Instead of sampling uniformly at random, we could also sample in proportion to the frequency with which the different values for dimension d are observed in the training data set. We then use a reconstruction loss as shown below where $\mathbf{v}'_n$ is the noisy copy of $\mathbf{v}_n$.

$$L(\mathbf{D}, \theta) = \sum_{n=1}^N \|\mathbf{v}_n - f_\theta(\mathbf{v}'_n)\|_2^2$$

10. (10 points) Variational Autoencoders: Suppose we build a standard variational autoencoder model to perform dimensionality reduction on multi-modal data consisting of short video clips with corresponding audio. A single data case consists of a video clip $\mathbf{v} = [\mathbf{i}, \mathbf{a}]$ represented as a sequence of image frames $\mathbf{i} = [\mathbf{i}_1, \dots, \mathbf{i}_T]$ paired with a sequence of audio samples $\mathbf{a} = [\mathbf{a}_1, \dots, \mathbf{a}_L]$. The learned variational autoencoder enables us to predict a probability distribution over the latent code $\mathbf{z} \in \mathbb{R}^K$ given the input video clip $\mathbf{v}$. Suppose at deployment time we encounter a video that has no audio content. Can we use our trained variational autoencoder to produce a probability distribution over latent representation $\mathbf{z}$ given the image frames $\mathbf{i}$ only? Explain your answer.

Example Solution: A VAE performs inference using a recognition network that predicts the mean and standard deviation values for a normal distribution over each latent dimension z_k given the input, in this case $\mathbf{v}$, as shown below. This recognition network specifies a mean function $m_\phi(\mathbf{v})$ and a standard deviation function $s_\phi(\mathbf{v})$ that are parameterized by one or more neural networks.

$$q_\phi(z_k|\mathbf{v}) = \mathcal{N}(z_k; m_\phi(\mathbf{v})[k], s_\phi^2(\mathbf{v})[k])$$

If we have an incomplete $\mathbf{v}$ vector, there is no immediate way to perform the inference network computations since $m_\phi(\mathbf{v})$ and $s_\phi(\mathbf{v})$ can not be computed without specifying values for all of their input dimensions. We could learn a separate inference network that only takes $\mathbf{i}$ as input, but this does not satisfy the premise of the question, which asks if we can use the model trained on complete $\mathbf{v}$ vectors. We could try imputing zeros or some other value for $\mathbf{a}$, but this is likely to generate spurious outputs from the recognition network. Finally, we could try to solve this problem using an iterative imputation approach where we start by imputing zeros for the audio $\mathbf{a}_0 = 0$, setting $\mathbf{v}_0 = [\mathbf{i}, \mathbf{a}_0]$, running inference to obtain z_0 , running the generator to obtain generated audio $\mathbf{a}_1$, and repeating the process with $\mathbf{v}_1 = [\mathbf{i}, \mathbf{a}_1]$ and so on. This approach constructs a Markov chain that approximately draws samples from $q_\phi(\mathbf{z}_k|\mathbf{i})$ while only needing access to the network trained on complete inputs.